Alon BENNATHAN

# Encrypted Cloud Storage

*Cryptography Engineering — Final Project Report*

## Code Structure

The project is organized into three layers. The crypto/ layer contains the three cryptographic primitives: password.rs (Argon2id), keys.rs (HKDF-SHA256), and aead.rs (AES-256-GCM). It has no dependencies on any other layer, it only operates on byte slices. The storage/ layer contains vault.rs, which abstracts the server as a directory of encrypted blobs, and manifest.rs, which defines the encrypted file index. The client/ layer ties everything together: profile.rs handles persistence of the user profile, and session.rs orchestrates the full workflows (register, login, upload, download, delete).

The threat model assumes an honest-but-curious server: it stores blobs faithfully but will read everything it receives. The server is never given any key material. It only ever sees random UUIDs as filenames, AES-256-GCM ciphertexts as file contents, and a single encrypted blob (manifest.enc) as the file index, it learns nothing about real filenames, file contents, or file sizes.

The key lifecycle is as follows. At login, the user's password is verified against an Argon2id PHC hash stored in user_profile.json. Argon2id then produces 32 bytes of key material, which HKDF expands into two independent 256-bit keys: file_key (for file contents) and meta_key (for the manifest). Both keys live only in the Session struct in RAM and are zeroized when the session ends. No key material is ever written to disk.

## Function Analysis

### 1. hash_password and crypto/password.rs

```
pub fn hash_password(password: &str) -> Result<String> {
    let salt   = SaltString::generate(&mut OsRng);  // fresh random salt
    let argon2 = Argon2::default();                  // Argon2id, m=19MB, t=2, p=1
    let hash   = argon2.hash_password(password.as_bytes(), &salt)?;
    Ok(hash.to_string())  // PHC: $argon2id$v=19$m=...,t=...,p=...$<salt>$<hash>
}
```

**What it does:** converts a plaintext password into a self-contained PHC string embedding the algorithm parameters, the random salt, and the hash output. This string is the only password-related data written to disk.

**How it works:** Argon2id fills a 19 MB memory buffer with pseudo-random data derived from the password and salt (memory-hard phase), then iterates over it to produce the final hash. A fresh 16-byte salt is generated by OsRng for every call.

**Why it is secure:** the memory-hardness limits GPU brute-force to a few hundred guesses per second per device. The random per-user salt prevents rainbow table attacks: two users with the same password produce completely different hashes. The comparison in verify_password runs in constant time (handled internally by the argon2 crate), preventing timing side-channel attacks.

### 2. derive_session_keys and crypto/keys.rs

```
pub fn derive_session_keys(ikm: &[u8], salt: &[u8]) -> Result<SessionKeys> {
    let file_key = derive_key(ikm, salt, b"ecs-file-encryption-v1")?;
    let meta_key = derive_key(ikm, salt, b"ecs-metadata-encryption-v1")?;
    Ok(SessionKeys { file_key, meta_key })
```

```
}
// derive_key: Hkdf::<Sha256>::new(Some(salt), ikm).expand(info, &mut okm)
```

**What it does:** from 32 bytes of Argon2 output, derives two independent 256-bit AES keys. file_key encrypts file contents; meta_key encrypts the manifest.

**How it works:** HKDF (RFC 5869) runs Extract (HMAC-SHA256 of the IKM with the salt, producing a PRK) then Expand (stretching the PRK with the info label to the desired length). The two info labels act as cryptographic namespaces.

**Why it is secure:** HKDF's security proof guarantees that outputs under different info labels are independent and learning file_key gives zero information about meta_key. The per-user HKDF salt (stored non-secretly in the profile) ensures two users with the same password derive completely different keys. SessionKeys implements Drop + zeroize, overwriting both keys in memory when the session ends.

## 3. encrypt / decrypt and crypto/aead.rs

```
// Wire format: [ nonce: 12 bytes | ciphertext | GCM tag: 16 bytes ]
pub fn encrypt(key: &[u8; 32], plaintext: &[u8]) -> Result<Vec<u8>> {
    let mut nonce_bytes = [0u8; 12];
    OsRng.fill_bytes(&mut nonce_bytes);      // fresh random nonce every call
    let ct = Aes256Gcm::new(...).encrypt(nonce, plaintext)?;
    Ok([nonce_bytes, ct].concat())
}
pub fn decrypt(key: &[u8; 32], data: &[u8]) -> Result<Vec<u8>> {
    let (nonce, ct) = data.split_at(12);
    Aes256Gcm::new(...).decrypt(nonce, ct)   // tag verified before returning plaintext
}
```

**What it does:** encrypt produces an authenticated ciphertext blob; decrypt recovers the plaintext and simultaneously verifies that the data has not been modified.

**How it works:** AES-256-GCM combines AES-CTR (confidentiality) with GHASH (a 128-bit authentication tag). The nonce is prepended in cleartext so decrypt can always recover it. The tag covers the entire ciphertext.

**Why it is secure:** the fresh random 96-bit nonce provides semantic security, encrypting the same file twice yields different ciphertexts, so the server cannot detect repeated uploads. The GCM tag ensures any modification to a stored blob (even one flipped bit) causes decrypt to fail before exposing any plaintext. This provides IND-CCA2 security. The 96-bit nonce space keeps the collision probability below $2^{-32}$ after $2^{32}$ encryptions.

## 4. upload and client/session.rs

```
pub fn upload(&self, local_path: &Path) -> Result<()> {
    let plaintext  = fs::read(local_path)?;
    let ciphertext = aead::encrypt(&self.keys.file_key, &plaintext)?;
    let storage_id = Uuid::new_v4().to_string();       // opaque UUID
    self.vault.store_blob(&storage_id, &ciphertext)?;  // server sees UUID only
    let mut manifest = self.vault.load_manifest(&self.keys.meta_key)?;
    manifest.upsert(filename, FileEntry { storage_id, size, uploaded_at });
    self.vault.save_manifest(&self.keys.meta_key, &manifest)?;
    Ok(())
}
```

**What it does:** encrypts a local file, stores it on the server under a random UUID, then updates the encrypted manifest to record the real filename → UUID mapping.

**How it works:** the file is encrypted with file_key and stored under a UUID v4. The manifest is loaded (decrypted with meta_key), updated with the new entry, and re-encrypted with a fresh nonce before being written back. The version counter in the manifest is incremented on every write.

**Why it is correct and secure:** the real filename never reaches the server, only the opaque UUID and the ciphertext. The manifest is protected by a separate key (meta_key), so an attacker who learns file_key cannot recover filenames. Re-encrypting the manifest with a fresh nonce on every write prevents the server from detecting whether the index changed between two operations.

## 5. save_manifest / load_manifest and storage/vault.rs

```
pub fn save_manifest(&self, meta_key: &[u8; 32], manifest: &Manifest) -> Result<()> {
    let json = serde_json::to_vec(manifest)?;
    let enc  = aead::encrypt(meta_key, &json)?;   // fresh nonce each write
    fs::write(self.root.join("manifest.enc"), enc)
}
pub fn load_manifest(&self, meta_key: &[u8; 32]) -> Result<Manifest> {
    let enc  = fs::read(self.root.join("manifest.enc"))?;
    let json = aead::decrypt(meta_key, &enc)?;    // tag verified here
    serde_json::from_slice(&json)
}
```

**What it does:** save_manifest serializes the in-memory file index to JSON, encrypts it with meta_key, and writes the blob to manifest.enc. load_manifest reverses this, decrypting and deserializing the manifest.

**How it works:** the Manifest struct holds a HashMap<filename, FileEntry> and a monotonically increasing version counter. Serialization produces a JSON byte string, which is fed directly into aead::encrypt. Because aead::encrypt generates a fresh random nonce, two consecutive saves of the same manifest produce different ciphertexts.

**Why it is correct and secure:** the GCM tag in load_manifest is verified before any data is returned, if the server tampers with manifest.enc, decryption fails immediately and no corrupted data is used. The version counter provides basic rollback detection: replacing manifest.enc with an older version produces a lower version number, which the application can detect. A known limitation is that full rollback protection would require a trusted monotonic counter (e.g., a hardware security module); the current scheme detects naive rollbacks only.